

Contents

Diagnóstico	1
Estructura, Operación, Azure DevOps, Repositorios y Ciclo de Desarrollo	1
Soporte Lógico SAS	1
1. Nomenclatura Interna	2
2. Contexto Estructural y Operativo	2
2.1 Modelo de Operación Técnica	2
2.2 Capacidad Técnica Disponible	3
2.3 Productos y Soluciones Soportadas	5
2.4 Líneas de Negocio Demandantes	5
2.5 Modelos de Despliegue Soportados	6
2.6 Modelos de Soporte y Atención	6
2.7 Hallazgo Estructural	7
3. Gestión de Demanda, Trazabilidad y Aarga Acumulada	7
3.1 Dimensionamiento de la Carga Acumulada	8
Ajuste de alcance — Azure DevOps subutilizado y segundo embudo FOMAG	9
3.2 Hallazgo de Gestión de Demanda	10
4. Ciclo actual de desarrollo y control operativo	11
4.1 Hallazgo del Ciclo Técnico	13
5. Estado actual del código, estructura técnica y retos de mantenimiento	13
5.1 Hallazgo del Estado Actual del Código	16
5.2 Gobierno del código, deuda propagada y diferenciación de problemas	17
6. Construcción, Paquetes y Despliegue	18
6.1 Hallazgo de Construcción, Paquetes y Despliegue	21
7. Estado actual de Azure DevOps	21
7.1 Hallazgo del estado actual de Azure DevOps	23
8. Estado actual de repositorios y ramas	24
8.1 Radiografía de repositorios principales	25
8.2 Hallazgo del estado actual de repositorios y ramas	26
9. Hallazgos consolidados	26
9.1 Lectura consolidada AS-IS	28
10. Conclusión AS-IS	28
Ajuste complementario — Flujo real de demanda, refinamiento y seguimiento operativo	29
A. Flujo actual real de ingreso y refinamiento de casos	29
B. Riesgo de duplicidad y solapamiento de historias	29
C. Precisión sobre Azure DevOps, Excel, chat y reuniones	30
D. Impacto del ciclo de dos meses	30
E. Hallazgo ajustado	30

Diagnóstico

Estructura, Operación, Azure DevOps, Repositorios y Ciclo de Desarrollo

Soporte Lógico SAS

Este documento presenta el estado actual del modelo operativo, el ciclo de desarrollo, la configuración de Azure DevOps, la gestión de repositorios y las prácticas de despliegue utilizadas actualmente en Soporte Lógico SAS. El análisis se basa en entrevistas con personas del equipo técnico, operativo y gerencial, así como en evidencia extraída de Azure DevOps, repositorios Git, exportaciones de Work Items y archivos de control interno en Excel.

Nota de alcance: este documento describe únicamente el estado actual . No incluye recomendaciones, responsables futuros ni planes de mejora.

1. Nomenclatura Interna

La organización utiliza términos propios para referirse a prácticas de desarrollo y operación que no siempre coinciden con la nomenclatura estándar de la industria. Esta diferencia puede generar ambigüedad en la comunicación con terceros técnicos, equipos operativos, proveedores o nuevos integrantes. Algunos ejemplos son:

Dominio	Término interno	Equivalencia funcional	Descripción
Metodología	Iteraciones	Sprints / ciclos de desarrollo	Ciclos de trabajo con duración aproximada de dos meses.
Metodología	Ponderación	Estimación ágil / Story Points	Forma interna y personalizada de estimar historias o actividades del backlog.
Código	Humano XX	Aplicación / módulo / API / componente	Se usa "Humano" como prefijo común para múltiples soluciones, módulos, servicios y componentes.
Código	Humano Fix	Release Branch	Rama usada para consolidar cambios antes de enviarlos a validación funcional o despliegue.
Despliegue	Ciclo versión X	Release Branch / Version Cutoff	Rama o corte usado para agrupar cambios de una versión que será desplegada.
Ambientes	Beta	SIT	Ambiente de pruebas internas de calidad.
Ambientes	Ciclo	UAT	Ambiente de validación funcional o aceptación de usuario.
Ambientes	Certificación	Staging	Ambiente de preproducción o certificación previa al pase productivo.

2. Contexto Estructural y Operativo

El modelo operativo actual concentra la demanda de distintas líneas de negocio, contratos y esquemas de atención sobre un mismo núcleo de producción técnica: Desarrollo, Calidad e Infraestructura.

2.1 Modelo de Operación Técnica

Aunque el organigrama formal divide a los equipos por funciones, en la práctica Desarrollo, Calidad, Infraestructura y apoyos externos actúan como capacidades operativas transversales para atender requerimientos evolutivos, bugs, soporte, urgencias, despliegues y ajustes derivados de clientes o contratos. Esta dinámica no se canaliza a través de un único embudo operativo. Durante el levantamiento se identifican al menos dos embudos visibles de demanda en Azure DevOps: el proyecto "Sistema de Información Humano", donde converge la demanda principal de varios productos y frentes, y el proyecto "FOMAG", usado de forma separada para historias de usuario y soporte de ese frente. Ambos proyectos presentan una configuración personalizada igual o equivalente. Esta fragmentación aumenta la complejidad para consolidar carga, avance, capacidad y trazabilidad de la fábrica.

Capacidad operativa	Liderazgo identificado	Rol y realidad en la práctica
Desarrollo y Arquitectura	Jonathan Moreno	Atiende la carga técnica transversal asociada a evolutivos, bugs y ajustes sobre el producto. También define lineamientos de arquitectura, prepara los paquetes ZIP para despliegue y entrega las instrucciones técnicas vía RFC a Infraestructura.
Calidad y Análisis (QA / BA)	Jenny Quintana	Articula actividades de análisis funcional y aseguramiento de calidad. Participa en la recepción, refinamiento y estructuración de requerimientos, así como en la validación previa de historias de usuario, bugs y ajustes que ingresan a los tableros de trabajo.
Infraestructura	Dayana González	Atiende infraestructura interna, soporte operativo, equipos, licenciamiento y habilitación de ambientes. También ejecuta despliegues hacia certificación o entrega a partir de los paquetes ZIP preparados por Desarrollo y las instrucciones técnicas recibidas.
Proveedor externo	David Suescun / Soin Soluciones	Apoya el desarrollo de tareas bajo demanda cuando la capacidad interna es insuficiente. Su participación no siempre opera integrada al flujo técnico de Desarrollo, lo que puede generar brechas de control, trazabilidad, alineación arquitectónica y entendimiento funcional de los requerimientos.

2.2 Capacidad Técnica Disponible

La operación depende de un grupo reducido de perfiles internos, aprendices y apoyo externo operativo. De acuerdo con el mapeo de capacidades del equipo, se identifican 13 personas asociadas directamente a la capacidad operativa de Desarrollo y Arquitectura, sin incluir el liderazgo del área. Esta capacidad combina perfiles internos, aprendices y dos apoyos externos operativos: un recurso provisto por la empresa complemento 360 y un apoyo independiente.

Adicionalmente, se identifica a David Suescun / Soin Soluciones como proveedor externo bajo un modelo de fábrica de software. Sin embargo, con la información disponible no se puede determinar con precisión su capacidad actual, dedicación efectiva, número de personas disponibles ni nivel de integración operativa con el equipo interno.

Dimensión observada	Lectura operativa
Tamaño y enrutamiento	La capacidad operativa directa está conformada por 13 personas asociadas a Desarrollo y Arquitectura, sin incluir el liderazgo del área. Este grupo absorbe demanda transversal de múltiples productos, bugs, evolutivos, soporte y despliegues. Adicionalmente, el tiempo real de desarrollo se ve reducido por actividades administrativas, reuniones, análisis, coordinación y soporte no siempre visibles en la medición operativa.
Coordinación y ceremonias diarias	Los dailies agrupan a más de 20 personas de Desarrollo, Calidad y Análisis. En una misma sesión se revisan múltiples proyectos, bugs, requerimientos y urgencias, con duración cercana a una hora. Esta dinámica reduce tiempo efectivo de ejecución, diluye el foco y no deja un reporte consolidado de avance, bloqueos o estado por proyecto.
Distribución de seniority	La capacidad registrada combina perfiles senior, semi-senior, intermedios, junior, aprendices y apoyo externo operativo. Esta heterogeneidad limita la elasticidad del equipo, porque actividades de arquitectura compleja, reportería profunda, integraciones, soporte especializado e incidentes críticos no pueden delegarse de forma uniforme.
Composición interna y externa	Se identifican perfiles internos, 2 aprendices SENA y 2 apoyos externos operativos: un recurso provisto por 360 y un apoyo independiente. Esta composición muestra que la capacidad efectiva de desarrollo no equivale al número total de personas, ya que no todos los perfiles tienen el mismo nivel de autonomía, especialización o disponibilidad para asumir trabajo crítico.
Silos de conocimiento	El dominio funcional y técnico del negocio está concentrado en personas específicas. El conocimiento profundo de componentes como Humano, Efinómina, Carrera, Humano Reportes, Humano API, Humano EP Java y Humano en Línea no se distribuye de manera homogénea en todo el equipo.
Capacidades críticas centralizadas	Labores determinantes para el ciclo de vida del software, como definición de arquitectura, integración de componentes, preparación de paquetes de despliegue, soporte especializado y resolución de incidentes complejos, recaen sobre perfiles puntuales.
Proveedor externo de fábrica	Se identifica a David Suescun / Soín Soluciones como proveedor externo bajo un modelo de fábrica de software. Sin embargo, con la información disponible no se puede determinar con precisión su capacidad actual, dedicación efectiva, número de personas disponibles ni nivel de integración operativa con el equipo interno.

Esta composición evidencia que la fábrica no solo enfrenta concentración de demanda sobre Desarrollo, Calidad e Infraestructura, sino también una capacidad técnica limitada, heterogénea y parcialmente incierta frente al volumen de productos, clientes y solicitudes que debe atender.

2.3 Productos y Soluciones Soportadas

La operación técnica no atiende un único producto homogéneo. Actualmente conviven varias soluciones con distintos niveles de dependencia funcional, técnica y evolutiva respecto al producto Humano.

Producto / Solución	Descripción operativa
Humano	Producto core de la compañía. Integra el back-office administrativo usado por áreas de Recursos Humanos para gestionar procesos de talento humano, y el portal de autogestión Humano en Línea, utilizado por empleados o funcionarios para consultar y gestionar información propia. Incluye módulos complementarios como Humano Reportes, junto con iniciativas nuevas como App e IA.
FOMAG	Back-office para usuarios de Fiduprevisora, orientado a la gestión transversal de procesos de prestaciones y afiliaciones del FOMAG sobre múltiples entidades. A diferencia de Humano, no opera sobre una sola entidad a la vez. Aunque debería comportarse como solución independiente, mantiene alto acoplamiento funcional y técnico con Humano, lo que dificulta su mantenimiento, versionamiento, trazabilidad y evolución.
Efinómina	Solución asociada al Consejo Superior de la Judicatura. Nació como una copia del ecosistema Humano y posteriormente fue modificada para operar con vida propia. Aunque hoy funciona como solución independiente, conserva herencias técnicas y funcionales del producto base, generando retos de mantenimiento, trazabilidad y control arquitectónico.
Carrera	Solución orientada a procesos de carrera, incluyendo componentes de calificación y selección.

Esta composición evidencia que la fábrica no gestiona únicamente variaciones del producto Humano, sino un ecosistema de soluciones relacionadas, algunas altamente acopladas al core y otras con independencia técnica significativa. Esta realidad incrementa la complejidad operativa cuando dichas soluciones convergen sobre las mismas capacidades técnicas, herramientas de seguimiento y flujos de entrega.

2.4 Líneas de Negocio Demandantes

Las siguientes líneas y frentes comerciales canalizan constantemente sus necesidades hacia el pool transversal de producción. Estas necesidades incluyen nuevos desarrollos, compromisos contractuales, estabilización, soporte, urgencias y ajustes específicos por cliente.

Línea / Frente	Responsable identificado
Sector Educativo / MEN	Edwin Gaona
SED Bogotá	Johan Miranda
FOMAG	María Victoria del Valle
Rama Judicial - Efinómina	Judith Gómez
Rama Judicial - Carrera	Ana María
Otros clientes públicos / privados	Edwin Gaona
Servicios especializados	Strey Vergara

Esta estructura evidencia que la demanda no proviene de un único producto o cliente, sino de varios frentes con prioridades, compromisos y tiempos de atención distintos.

2.5 Modelos de Despliegue Soportados

Actualmente conviven tres modelos de despliegue, lo que obliga a considerar particularidades por cliente, ambiente y contrato durante los procesos de preparación de entrega, liberación y soporte, incrementando la complejidad operativa.

Modelo	Descripción
SaaS / PaaS	Soluciones operadas o administradas desde infraestructura cloud.
On-Premise	Instalaciones físicas o virtuales sobre infraestructura propia del cliente.
BPO / Tercerización	Operación especializada o tercerizada según condiciones contractuales.

2.6 Modelos de Soporte y Atención

Actualmente conviven diferentes modelos de soporte y atención, con niveles de servicio, canales y expectativas distintas. En términos generales, se identifican dos categorías principales:

Modelo	Descripción
Soporte estándar	Conducto regular de atención para requerimientos con tiempos de espera tipificados.
Servicios especializados	Atención diferenciada para clientes específicos, incluyendo soporte directo, atención VIP, asistencia en sitio y desarrollos a medida.

Detalle Operativo de Servicios Especializados:

Elemento	Descripción
Producto principal	Humano
Alcance	Soporte directo, atención VIP, asistencia en sitio y desarrollos a medida.
Clientes aproximados	~18 entidades
Clientes	Departamento del Cesar, Gobernación de Cundinamarca y Municipio de Pitalito, entre otros.
Liderazgo identificado	Strey Vergara

2.7 Hallazgo Estructural

Múltiples productos, líneas de negocio, clientes, modelos de soporte y modelos de despliegue convergen sobre las mismas capacidades técnicas transversales: Desarrollo, Calidad e Infraestructura. Esta condición genera concentración de demanda sobre el núcleo de producción técnica, pero no a través de un único embudo operativo. Durante el levantamiento se identifican al menos dos embudos visibles en Azure DevOps: el proyecto “Sistema de Información Humano” y el proyecto “FOMAG”. Esta fragmentación incrementa la complejidad operativa y reduce la visibilidad consolidada por producto, cliente o frente de trabajo. Adicionalmente, la capacidad operativa directa de Desarrollo y Arquitectura se soporta en un grupo reducido de perfiles internos, aprendices y apoyos externos operativos, con distintos niveles de seniority, autonomía y dominio funcional. Esto limita la elasticidad del equipo para absorber simultáneamente evolutivos, bugs, soporte, urgencias, estabilización y despliegues.

En el caso del proveedor externo de fábrica, David Suescun / Soin Soluciones, no se cuenta con visibilidad suficiente sobre su capacidad real disponible, dedicación efectiva, número de personas asignadas ni nivel de integración operativa con el equipo interno. Adicionalmente, parte de la demanda de FOMAG es gestionada en un proyecto separado de Azure DevOps, configurado de forma personalizada igual o equivalente al proyecto “Sistema de Información Humano”. Sin embargo, sus Work Items no fueron analizados en detalle dentro del alcance actual. Por esta razón, ni la capacidad del proveedor ni la carga real de FOMAG pueden considerarse completamente dimensionadas dentro del modelo operativo actual. En la práctica, la fábrica sostiene la operación mediante un modelo transversal que le permite responder a distintos frentes, pero que también diluye la separación entre soporte, evolución, estabilización y entrega, aumentando la dependencia de personas clave y la presión sobre la capacidad técnica disponible.

3. Gestión de Demanda, Trazabilidad y Aarga Acumulada

Azure DevOps funciona como plataforma formal para registrar la demanda. El hallazgo no es ausencia de herramienta, sino subutilización de Azure DevOps como sistema integral de gobierno operativo. La operación registra historias, bugs, tareas y casos dentro de la herramienta; sin embargo, la clasificación, priorización, estimación, trazabilidad de cierre, medición de capacidad, separación de demanda planificada versus operación diaria y control de proveedores no se explotan de forma homogénea.

En la práctica, la demanda se observa en al menos dos embudos de gestión: el proyecto “Sistema de Información Humano” y el proyecto “FOMAG”. Ambos proyectos están configurados con una personalización igual o equivalente. El primero fue analizado cuantitativamente dentro del alcance inicial; el segundo ya permite identificar una carga parcial adicional de 244 historias pendientes administradas por David Suescun / Soin Soluciones como proveedor externo. Además, la clasificación por cliente, producto o frente depende de convenciones manuales, archivos externos y controles complementarios que dificultan consolidar métricas confiables sobre carga, avance, esfuerzo real y capacidad instalada. Esta limitación no solo afecta el seguimiento operativo; también dificulta dimensionar con precisión la carga acumulada de trabajo pendiente y la capacidad requerida para atenderla.

Aspecto observado	Descripción	Impacto operativo
Embudos de demanda en Azure DevOps	Se identifican al menos dos proyectos usados como embudos de gestión: “Sistema de Información Humano” y “FOMAG”. El primero concentra la demanda principal de múltiples productos, clientes y frentes. El segundo se usa de forma separada para historias de usuario y soporte de FOMAG. Ambos presentan una configuración personalizada igual o equivalente.	La demanda queda fragmentada entre proyectos separados. El análisis cuantitativo se realizó sobre “Sistema de Información Humano”, por lo que la carga de FOMAG no queda incluida en los totales analizados.

Aspecto observado	Descripción	Impacto operativo
Clasificación manual por cliente, producto o frente	No todos los clientes, productos o frentes se encuentran parametrizados de forma homogénea en Azure DevOps. Por esta razón, la operación recurre a convenciones manuales, como incluir el nombre del cliente o frente al inicio del título de historias, bugs o requerimientos.	La trazabilidad depende de la disciplina manual de quien registra la información. Esto reduce la confiabilidad de filtros, reportes automáticos y métricas por cliente o producto.
Uso de archivos Excel paralelos	Para complementar las limitaciones de clasificación y seguimiento en Azure DevOps, la operación utiliza archivos paralelos de Excel. Estos archivos permiten controlar asignaciones específicas, seguimiento por cliente, avances operativos y versiones de paquetes.	Se genera doble registro, fragmentación de información y dependencia de actualización manual. Esto dificulta consolidar una única fuente de verdad sobre el estado real de la demanda.
Estimación, ponderación y visibilidad de carga	Actualmente no se evidencia un uso consistente de estimaciones de esfuerzo en Azure DevOps. Parte de la medición operativa se apoya en mecanismos externos, como ponderaciones o controles manuales, que no siempre reflejan la carga completa del trabajo.	Se dificulta medir el esfuerzo real de los equipos, especialmente al considerar actividades administrativas, reuniones, coordinación operativa, análisis, validación, reprocesos y soporte no planificado.
Ausencia de KPIs operativos en Azure DevOps	Aunque Azure DevOps funciona como plataforma formal de registro, no se evidencia una configuración consolidada de indicadores operativos que permita visualizar avance por proyecto, producto, cliente, frente de trabajo o iteración. Indicadores como avance comprometido vs. ejecutado, carga activa por equipo, bugs abiertos, elementos bloqueados, envejecimiento del backlog, throughput o cumplimiento por iteración no se encuentran disponibles de forma homogénea para el equipo.	La falta de métricas visibles limita la lectura objetiva del avance y obliga a compensar el seguimiento mediante reuniones extensas, conversaciones manuales, archivos externos y conocimiento concentrado en líderes o personas clave.

3.1 Dimensionamiento de la Carga Acumulada

La falta de segmentación confiable por producto, cliente o frente limita la capacidad de analizar automáticamente la demanda pendiente. Aunque se descargaron las historias de usuario, bugs y tareas disponibles desde el proyecto “Sistema de Información Humano” en Azure DevOps, la clasificación actual no permite distinguir de forma simple y consistente qué carga corresponde a cada solución, cliente o línea operativa.

Aun con esta limitación, las exportaciones del proyecto “Sistema de Información Humano” permiten

dimensionar parcialmente la magnitud del trabajo acumulado. Al excluir elementos cerrados, removidos o rechazados, se identifican aproximadamente 447 historias de usuario, 268 bugs y 138 tareas en estados abiertos o activos. Este conteo no incluye las historias de desarrollo, soporte, bugs o tareas de FOMAG gestionadas en el proyecto separado de Azure DevOps.

Tipo de Work Item	Total exportado	Pendiente / activo aproximado
User Stories	1.609	447
Bugs	1.130	268
Tasks	2.887	138
Total	5.626	853

Solo entre historias de usuario y bugs se observan aproximadamente 715 elementos pendientes o activos en el proyecto “Sistema de Información Humano”. Esto representa más de 400 solicitudes funcionales o evolutivas por realizar y más de 250 errores pendientes de corrección o atención. Parte de esta carga corresponde a elementos antiguos, algunos con varios años de permanencia en el backlog.

Ajuste de alcance — Azure DevOps subutilizado y segundo embudo FOMAG

El hallazgo no es ausencia de Azure DevOps. La herramienta se usa en la operación actual, pero no se explota al 100% como fuente única de gobierno, medición y decisión. Esto obliga a complementar la lectura operativa con convenciones manuales, tableros personalizados, conocimiento de personas clave, reuniones y controles paralelos.

Además del proyecto “Sistema de Información Humano”, se identifica un segundo embudo operativo en Azure DevOps asociado a FOMAG. Este tablero es administrado por David Suescun / Soín Soluciones como proveedor externo y presenta una configuración personalizada similar o equivalente a la del tablero principal.

Con la información actual, se identifican al menos 244 historias de usuario pendientes de hacer en FOMAG. Adicionalmente, este mismo tablero gestiona errores dentro del flujo operativo, pero dichos errores no se encuentran todavía cuantificados dentro del presente diagnóstico.

Esta condición modifica la lectura inicial de carga acumulada. Los 715 elementos abiertos identificados en “Sistema de Información Humano” no representan la demanda total visible de la fábrica. Al incorporar las 244 historias pendientes de FOMAG, la línea base mínima conocida asciende a 959 elementos visibles, sin contar errores de FOMAG no cuantificados, nueva demanda, incidentes diarios, soporte operativo, estabilizaciones ni reprocesos.

Embudo operativo	Demanda visible	Observación
Sistema de Información Humano	715 elementos entre HU y bugs	Incluye 447 historias y 268 bugs abiertos / activos.
FOMAG	244 historias pendientes	No incluye errores del mismo tablero, pendientes de cuantificar.
Total mínimo conocido	959 elementos	Línea base parcial, no equivalente a demanda total de la fábrica.

Aun bajo un escenario optimista de cierre de un elemento por día, esta demanda visible representa entre 2 y 3 años de trabajo calendario. Si se calcula sobre días hábiles y se incorporan errores de FOMAG, incidentes diarios, soporte operativo y nueva demanda, la ventana real de atención puede ser mayor.

El punto crítico no es únicamente el volumen acumulado, sino la falta de una lectura confiable de capacidad efectiva. La capacidad nominal del equipo interno no equivale a capacidad efectiva para

absorber desarrollo complejo, soporte especializado, incidentes críticos, estabilización y evolución funcional en paralelo. Adicionalmente, la capacidad instalada del proveedor FOMAG no se encuentra completamente dimensionada, ya que no se cuenta con una lectura suficiente de su velocidad, dedicación efectiva, número real de recursos, carga correctiva y predictibilidad de entrega.

El punto crítico es que la carga acumulada cuantificada en “Sistema de Información Humano” convive con la entrada continua de nueva demanda comercial, contractual y operativa, y además con demanda no cuantificada en el proyecto “FOMAG”. Mientras la compañía sigue incorporando nuevos compromisos, servicios especializados, ajustes por cliente y evolutivos, el mismo pool transversal debe absorber simultáneamente backlog pendiente, bugs, soporte, urgencias, estabilización y nuevos desarrollos.

En términos operativos, la fábrica enfrenta una brecha entre la demanda comprometida y la capacidad disponible para atenderla. Dado que no se evidencia un uso consistente de estimaciones de esfuerzo en Azure DevOps, esta carga no puede traducirse automáticamente en capacidad requerida. Sin embargo, por el volumen de historias, bugs y tareas abiertas en “Sistema de Información Humano”, y considerando el ritmo histórico de atención observado en la operación, la demanda pendiente de este proyecto podría representar una ventana aproximada de entre 15 y 21 meses de trabajo, sin considerar el ingreso de nuevas solicitudes ni la carga no cuantificada del proyecto “FOMAG”.

Si el ritmo de entrada de nueva demanda continúa sin una ampliación proporcional de capacidad técnica ni una mejora en la trazabilidad, el backlog tenderá a crecer, afectando tiempos de respuesta, predictibilidad de entrega, calidad del producto y sostenibilidad del equipo. Esta lectura debe considerarse parcial, porque no incorpora la carga activa del proyecto “FOMAG”.

3.2 Hallazgo de Gestión de Demanda

La gestión de demanda se encuentra fragmentada entre proyectos de Azure DevOps, convenciones manuales y archivos Excel paralelos. No se evidencia un único embudo de entrada para toda la demanda: se identifica un embudo principal en el proyecto “Sistema de Información Humano” y un segundo embudo separado en el proyecto “FOMAG”. Ambos proyectos presentan una configuración personalizada igual o equivalente, pero el análisis detallado de Work Items se realizó únicamente sobre “Sistema de Información Humano”.

Esta situación limita la generación de métricas confiables por cliente, producto, frente de negocio o tipo de demanda. Además, impide dimensionar con precisión la capacidad requerida para atender la carga total de la fábrica. La carga acumulada identificada supera los 700 elementos abiertos entre historias de usuario y bugs en “Sistema de Información Humano”, y al incorporar las 244 historias pendientes de FOMAG asciende a una línea base mínima conocida de 959 elementos visibles.

Esta lectura debe entenderse como parcial: no incorpora errores de FOMAG no cuantificados, incidentes diarios, soporte operativo, nueva demanda, estabilizaciones ni reprocesos. Aun bajo un escenario optimista de cierre de un elemento por día, la demanda visible actual representa entre 2 y 3 años de trabajo calendario; sobre días hábiles, la ventana real puede ser mayor.

La brecha ejecutiva no es que Azure DevOps no exista. La brecha es que se encuentra subutilizado como sistema de gobierno operativo, lo que limita medir con suficiente confiabilidad demanda, capacidad, prioridad, velocidad, trazabilidad de cierre, trabajo no planificado y control de proveedores.

Como resultado, la operación mantiene una presión creciente sobre Desarrollo, Calidad e Infraestructura, mientras continúa ingresando nueva demanda comercial, contractual y operativa a los embudos de trabajo identificados.

Sí. El punto está muy bien, pero para dejarlo más KISS, YAGNI y DRY yo haría tres ajustes:

1. Unificaría filas que dicen casi lo mismo.
2. Sacaría del punto 4 lo que pertenece más al punto 5, como versión desplegada o entrega final.
3. Mantendría el foco del punto 4 en el flujo técnico hasta beta, ciclo y certificación.

La versión más limpia la dejaría así:

4. Ciclo actual de desarrollo y control operativo

El ciclo actual de desarrollo combina planificación en Azure DevOps, seguimiento operativo en archivos Excel, trabajo por ramas, integración mediante Pull Requests, validaciones en ambientes beta, cierre de ciclo y pase a certificación. Aunque existen herramientas formales para gestionar trabajo y código fuente, el flujo real depende de controles paralelos, coordinación manual, conocimiento del equipo y validaciones operativas entre Desarrollo, Calidad e Infraestructura.

Aspecto observado	Descripción	Impacto operativo
Iteraciones amplias y demanda mixta	El trabajo se organiza mediante iteraciones con duración aproximada de dos meses. En estas ventanas conviven historias de usuario, bugs, ajustes, requerimientos evolutivos, soporte contractual, estabilización y urgencias operativas.	La iteración funciona como un contenedor amplio de demanda, lo que reduce la predictibilidad y facilita que urgencias o compromisos externos desplacen actividades previamente planificadas.
Seguimiento fragmentado entre Azure DevOps y Excel	Aunque Azure DevOps funciona como herramienta formal de registro, el seguimiento operativo se complementa con archivos Excel separados por cliente, proyecto o frente de trabajo. Algunos controles específicos, como asignaciones, avances y validaciones, no se gestionan de forma centralizada en Azure DevOps.	Se genera doble registro, dependencia de actualización manual y dificultad para consolidar una única fuente de verdad sobre el estado real del trabajo.
Clasificación manual de clientes o frentes	Cuando un cliente o frente no está parametrizado formalmente en Azure DevOps, el equipo recurre a convenciones manuales, como escribir el nombre del cliente al inicio del título del Work Item o filtrar por la persona que creó el caso.	La trazabilidad depende de la disciplina manual de registro, lo que reduce la confiabilidad de filtros, consultas, tableros y métricas automáticas por cliente, producto o frente.
Estimación y carga no visible	El esfuerzo se estima mediante una ponderación interna. Por ejemplo, una ponderación 7 puede equivaler aproximadamente a 10 días de desarrollo. Sin embargo, esta medición no siempre incorpora reuniones, soporte, coordinación, revisiones, reprocesos, validaciones o atención de urgencias.	La estimación permite una referencia operativa, pero puede no reflejar la capacidad real disponible ni el esfuerzo completo requerido para desarrollar, validar, corregir e integrar el trabajo.
Seguimiento diario amplio	El seguimiento de la iteración se apoya en reuniones diarias donde se revisan simultáneamente múltiples proyectos y frentes de trabajo. Estas sesiones pueden involucrar más de 20 personas y extenderse cerca de una hora.	Aunque permiten visibilidad conversacional, su duración, número de participantes y ausencia de una salida consolidada limitan su efectividad como mecanismo de seguimiento operativo por proyecto.

Aspecto observado	Descripción	Impacto operativo
Desarrollo por ramas	Los desarrolladores trabajan sobre ramas asociadas a requerimientos, bugs, fixes, ajustes o ciclos de versión, generalmente a partir de ramas base de integración como humano-develop u otras equivalentes según el repositorio.	La coexistencia de múltiples tipos de ramas incrementa la complejidad para determinar qué cambios hacen parte efectiva de una entrega.
Integración mediante Pull Request	Una vez finalizado el cambio, el desarrollador crea un Pull Request hacia la rama base de integración, como humano-develop. La aprobación depende de revisión técnica manual, usualmente por parte de un líder técnico.	El Pull Request introduce un punto formal de control, pero la integración depende de revisión, aprobación y coordinación manual. No se evidencia un esquema homogéneo de integración continua aplicado de forma consistente sobre todos los productos, repositorios y componentes.
Trazabilidad del cambio	No se evidencia una trazabilidad homogénea entre Work Item, rama de desarrollo, Pull Request, validación beta y cierre de ciclo.	Se dificulta reconstruir rápidamente qué cambio corresponde a qué requerimiento, en qué rama fue desarrollado, cuándo se integró, cómo fue validado y si quedó incluido en una versión específica.
Criterios de avance y cierre	No se evidencia una definición homogénea de criterios de entrada, salida y terminado para mover un cambio entre desarrollo, integración, beta, cierre de ciclo y certificación.	Puede existir diferencia entre “desarrollado”, “integrado”, “probado”, “aprobado” y “listo para entrega”, dificultando una lectura clara del estado real del trabajo.
Control de calidad técnica del código	No se evidencia un control automatizado y homogéneo de calidad técnica del código integrado al flujo de desarrollo, Pull Requests o ramas de integración.	La revisión técnica depende principalmente de validación manual, criterio del revisor y conocimiento del equipo, lo que limita la detección temprana de deuda técnica, vulnerabilidades, duplicación o incumplimiento de estándares antes de avanzar hacia beta o certificación.
Pruebas beta en ambientes diferenciados	Después de la integración, los cambios pasan a validación en ambientes beta. Estas validaciones pueden requerir que el desarrollador baje cambios, recompile e instale manualmente en servidores específicos. Conviven ambientes con configuraciones diferenciadas por motor de base de datos, cliente o infraestructura, como Beta Oracle, Beta SQL o ambientes específicos por cliente.	La validación depende de coordinación manual, disponibilidad de ambientes, configuración correcta y conocimiento técnico del componente involucrado. Esto incrementa la complejidad para identificar dónde debe probarse cada cambio y qué condiciones pueden afectar la validación funcional.

Aspecto observado	Descripción	Impacto operativo
Retornos por defectos o ajustes	Cuando se identifican defectos o ajustes durante beta, el flujo retorna a Desarrollo para corrección, integración y nueva validación.	El ciclo puede repetirse varias veces antes de que el cambio quede listo para cierre o certificación, aumentando reprocesos y dependencia de coordinación entre equipos.
Bloqueos y dependencias operativas	El avance del cambio depende de coordinación entre Desarrollo, Calidad y, en algunos casos, Infraestructura para habilitar ambientes, validar componentes o resolver ajustes de configuración.	Los bloqueos no siempre quedan visibles como información estructurada del ciclo, lo que dificulta medir tiempos de espera, causas de retraso y dependencias reales entre equipos.
Cierre de ciclo y pase a certificación	Al acercarse el cierre de la iteración o versión, los cambios aprobados se consolidan en ramas de ciclo y pueden combinarse con ramas de estabilización como humano fix. Luego se llevan hacia una rama o ambiente de certificación, como humano certificación, para validación previa a la entrega formal.	El cierre exige aislar manualmente el conjunto de cambios que hará parte de la versión, lo que aumenta la dependencia de convenciones operativas, merges manuales y conocimiento técnico del equipo.

4.1 Hallazgo del Ciclo Técnico

El ciclo de desarrollo actual es funcional, pero depende en gran medida de orquestación humana para mover, integrar, compilar y validar el código entre etapas. Aunque existen repositorios, ramas de trabajo y Pull Requests, no se evidencia un flujo homogéneo de integración continua que permita automatizar de forma consistente el tránsito del software hacia beta, ciclo y certificación. En la práctica, el avance exige coordinación manual, descarga de cambios, recompilación e instalación en ambientes específicos según componente, cliente, configuración o motor de base de datos. Esta condición aumenta la probabilidad de reprocesos, falsos positivos en pruebas de Calidad, demoras en la estabilización de versiones y dependencia de líderes técnicos o personas clave para consolidar y validar los cambios.

Adicionalmente, no se evidencia un control automatizado y homogéneo de calidad técnica del código dentro del flujo de integración, lo que mantiene la revisión dependiente del criterio manual y del conocimiento del equipo. En consecuencia, la organización logra sostener el ciclo técnico, pero con baja automatización, trazabilidad parcial y una carga operativa alta para saber qué cambios fueron integrados, probados, corregidos y efectivamente incluidos en una versión.

5. Estado actual del código, estructura técnica y retos de mantenimiento

Humano no es una sola aplicación simple, sino un ecosistema de soluciones, módulos y proyectos relacionados. Su estructura actual combina aplicación web principal, módulos funcionales, capas de acceso a datos, capas de proceso, componentes transversales, reportería, servicios, bases de datos y repositorios complementarios.

Esta composición permite sostener una operación amplia y funcional, pero también incrementa la complejidad para entender, modificar, probar, compilar y evolucionar el producto. En la práctica, trabajar sobre Humano exige conocer relaciones entre módulos, dependencias entre proyectos, configuraciones específicas, tecnologías de distintas generaciones y patrones históricos de implementación.

Aspecto observado	Descripción	Impacto operativo
Humano como ecosistema técnico central	Humano opera como producto core de la compañía, pero no como una aplicación única. Su estructura agrupa soluciones, módulos funcionales, capas técnicas, componentes transversales, reportería, servicios y bases de datos.	El producto requiere conocimiento del ecosistema completo para intervenirlo con seguridad. Un cambio aparentemente puntual puede involucrar múltiples capas, referencias o componentes relacionados.
Arquitectura modular histórica	Humano Fase I funciona como una aplicación web principal sobre .NET Framework e IIS. Alrededor de esta fachada conviven módulos funcionales completos, módulos satélites y componentes transversales.	La organización por módulos ayuda a separar dominios funcionales, pero la cantidad de proyectos y dependencias aumenta la dificultad para entender responsabilidades, límites e impactos.
Módulos compartidos y acoplamiento con FOMAG	FOMAG se apoya en módulos y componentes que conviven dentro del ecosistema Humano. Su operación comparte estructura, convenciones y base técnica con el producto central.	La separación entre Humano y FOMAG no es completamente limpia desde el punto de vista técnico. Esto dificulta aislar cambios, validar impactos y evolucionar cada frente de manera independiente.
Preparación del entorno de desarrollo	Trabajar sobre Humano no parte de abrir una única solución aislada. El entorno requiere identificar componentes, soluciones, referencias, configuraciones y relaciones entre proyectos del ecosistema.	El onboarding técnico depende de conocimiento específico del producto. Aumenta la curva de aprendizaje y el riesgo de modificar, compilar o validar un componente sin entender sus dependencias reales.
Crecimiento por copia, variación y acumulación	La evolución del código evidencia un patrón histórico de extensión de módulos, creación de variantes, reutilización de estructuras existentes y acumulación de componentes. Este patrón no se limita a reportería; corresponde a una forma general de crecimiento del producto.	Se incrementa el volumen de código, la duplicidad funcional, la dificultad para aislar cambios y el riesgo de corregir un caso sin corregir sus variantes relacionadas.
Convivencia tecnológica	En el ecosistema conviven componentes contruidos sobre distintas generaciones tecnológicas, incluyendo .NET Framework, aplicaciones sobre IIS, servicios SOAP, componentes Java asociados a reportes o procesos, componentes .NET Core y desarrollos más recientes sobre .NET 8.	El equipo debe dominar múltiples modelos de compilación, configuración, ejecución y diagnóstico. Esto aumenta la dependencia de personas con conocimiento histórico.

Aspecto observado	Descripción	Impacto operativo
Dependencias transversales	Existen componentes compartidos, como utilidades comunes, procesos generales, capas de base de datos y motor de reportes, que pueden ser usados por varios módulos o flujos funcionales.	Los cambios sobre componentes comunes pueden generar impacto colateral en distintos módulos, clientes o procesos del producto.
Patrones repetidos de acceso a datos	Se identifican consultas reutilizadas o repetidas en distintos puntos del código. Un caso observado es la consulta sobre CALENDARIODET_TH, presente en 9 clases y 15 métodos, incluyendo usos en componentes de Humano y FOMAG.	La repetición de consultas en múltiples clases dificulta estandarizar el comportamiento, optimizar el acceso a datos y corregir de forma centralizada posibles problemas de rendimiento o consistencia funcional.
Consultas dentro de ciclos de negocio	Un caso representativo es DíasDescansoRemunerado, donde la lógica evalúa días posteriores a una fecha de retiro y consulta la base de datos dentro de un ciclo.	En procesos masivos, este patrón puede multiplicar consultas, conexiones y viajes a base de datos, afectando rendimiento, escalabilidad y mantenibilidad.
Lógica distribuida entre aplicación y base de datos	Parte de la lógica se resuelve en C# y parte mediante consultas pequeñas y repetidas contra la base de datos. En algunos casos, las consultas pueden no incorporar todos los filtros funcionales esperados, como calendario o contexto.	Se dificulta optimizar, probar y razonar sobre el comportamiento completo del cálculo. También puede aumentar el riesgo de ambigüedad funcional o resultados inconsistentes.
Presión observable en base de datos	Los reportes de rendimiento de base de datos muestran síntomas de carga relevante, como sentencias SQL principales, esperas de E/S, lecturas y eventos asociados a confirmaciones. Esta evidencia no atribuye la carga a un único método, pero sí refuerza la importancia de revisar cómo el código consulta la base de datos.	La calidad del código de acceso a datos incide directamente en desempeño, estabilidad y capacidad de diagnóstico del producto.

Aspecto observado	Descripción	Impacto operativo
Gobierno del código y reglas de escritura	No se evidencia un estándar institucional, homogéneo y verificable para la escritura, organización y revisión del código fuente. Aunque existen prácticas individuales y revisión manual mediante Pull Requests, no se observa una guía formal aplicada de manera consistente sobre nomenclatura, estructura de clases, separación de responsabilidades, manejo de dependencias, acceso a datos, tratamiento de errores, logging, pruebas automatizadas, criterios de refactorización o deuda técnica aceptable.	La calidad y mantenibilidad del código dependen en buena medida del criterio individual, del conocimiento histórico y de la revisión manual. Esto dificulta aplicar controles automáticos, acelerar onboarding y establecer criterios objetivos de aceptación técnica antes de integrar cambios.
Crecimiento por copia, adaptación y variación	La evolución del ecosistema evidencia reutilización de estructuras, módulos, consultas, reportes y convenciones existentes para construir o extender soluciones relacionadas. En algunos casos, las soluciones derivadas no nacen desde una separación arquitectónica completamente limpia, sino desde adaptación del producto base y sus componentes.	El patrón permitió responder rápidamente a necesidades funcionales y contractuales, pero también propagó complejidad, duplicidad, acoplamiento y reglas dispersas hacia otros productos, módulos o repositorios.
Propagación de deuda técnica	Parte de la deuda técnica no queda contenida en Humano. Al copiar, adaptar o extender componentes hacia FOMAG, Efinómina, Carrera, reportería, APIs o módulos complementarios, también se trasladan patrones de implementación, dependencias y problemas de mantenibilidad.	Corregir un problema en un frente no garantiza corregir sus variantes en otros componentes. La fábrica enfrenta deuda técnica distribuida, con mayor esfuerzo de análisis, validación y control de impacto.

5.1 Hallazgo del Estado Actual del Código

Humano es un ecosistema técnico funcional y maduro, pero con alta complejidad estructural y de mantenimiento. No opera como una aplicación simple, sino como un conjunto de soluciones, módulos, capas, componentes transversales, tecnologías y configuraciones que han evolucionado durante años sobre una base común.

La principal condición observada es que el producto ha crecido por acumulación histórica. Esta evolución ha permitido sostener el negocio y atender múltiples necesidades funcionales, contractuales y operativas, pero también ha incrementado el acoplamiento, la duplicidad, la curva de aprendizaje y el riesgo de impacto colateral.

El acoplamiento con FOMAG refuerza esta condición, porque parte de sus capacidades conviven dentro del ecosistema Humano y comparten módulos, referencias o componentes del producto central. Esto dificulta separar con claridad qué pertenece al core, qué pertenece a una solución específica y qué

impacto puede tener un cambio sobre otros frentes.

A nivel de código, se observan patrones repetidos de acceso a datos y consultas distribuidas en múltiples clases y métodos. El caso de CALENDARIODET_TH, identificado en 9 clases y 15 métodos, evidencia que algunas reglas o consultas de negocio no están centralizadas, lo que dificulta su optimización, corrección y gobierno técnico.

En síntesis, el código actual permite sostener la operación, pero presenta retos importantes de gobernabilidad técnica: acoplamiento entre módulos, crecimiento por copia y variación, convivencia tecnológica, trazabilidad limitada de dependencias y patrones de acceso a datos que pueden afectar rendimiento y mantenibilidad.

5.2 Gobierno del código, deuda propagada y diferenciación de problemas

Durante la revisión del ecosistema técnico no se evidencia un estándar institucional, homogéneo y verificable para la escritura, organización y revisión del código fuente. Aunque existen prácticas individuales, conocimiento técnico del equipo y revisión manual mediante Pull Requests, no se observa una guía formal aplicada de manera consistente sobre nomenclatura, estructura de clases, separación de responsabilidades, manejo de dependencias, acceso a datos, tratamiento de errores, logging, pruebas automatizadas, criterios de refactorización o definición de deuda técnica aceptable.

Esta condición ayuda a explicar parte del crecimiento por acumulación observado en el producto. La evolución histórica del código parece haberse orientado principalmente a resolver necesidades funcionales, contractuales y operativas, pero sin un marco técnico suficientemente uniforme para controlar la forma en que dichas soluciones se incorporaban al ecosistema.

Adicionalmente, el ecosistema evidencia un patrón histórico de crecimiento por copia, adaptación y variación de componentes existentes. Algunas soluciones no parecen haber nacido desde una separación arquitectónica completamente limpia, sino a partir de reutilización, duplicación o adaptación del producto base Humano. Este patrón permitió responder rápidamente a necesidades de negocio, nuevos contratos y frentes funcionales, pero también propagó problemas técnicos del producto original hacia soluciones derivadas como FOMAG, Efinómina, Carrera, reportería, APIs y componentes complementarios.

Como consecuencia, la deuda técnica no solo se acumuló; también se propagó. Esto significa que el problema no está concentrado únicamente en un repositorio o producto, sino distribuido en varios módulos, reportes, reglas de negocio, consultas, configuraciones y prácticas de despliegue. Corregir un problema en el producto base no garantiza corregir sus variantes copiadas o adaptadas en otros componentes.

Para evitar mezclar síntomas distintos, el AS-IS debe diferenciar cuatro frentes que están relacionados, pero tienen naturaleza diferente:

Elemento observado	Naturaleza del problema	Lectura AS-IS
Backlog acumulado	Gestión de demanda y capacidad	Existen historias, bugs y tareas pendientes que superan la capacidad visible de atención. Este frente refleja presión operativa, necesidad de priorización y trazabilidad insuficiente de la demanda.
Ramas acumuladas	Gobierno de configuración y versionamiento	Existen múltiples ramas históricas, personales, funcionales, de bug, fix, ciclo y despliegue. Este frente refleja dificultad para controlar integraciones, depurar histórico y auditar releases.

Elemento observado	Naturaleza del problema	Lectura AS-IS
Código heterogéneo o desordenado	Gobierno técnico, arquitectura interna y mantenibilidad	Se evidencian patrones repetidos, reglas dispersas, acoplamiento, variantes funcionales y ausencia de estándares verificables de escritura y organización del código. Este frente refleja deuda técnica estructural.
Construcción manual de productos	Build, empaquetamiento y delivery	La generación de entregables depende de recompilación manual, paquetes ZIP e instrucciones operativas. Este frente refleja bajo nivel de automatización, versionamiento técnico y trazabilidad de artefactos.

Esta separación es relevante porque cada frente requiere un tratamiento distinto. La depuración de ramas no resuelve por sí sola la heterogeneidad del código; la automatización de builds no elimina el backlog acumulado; y la definición de estándares de codificación no corrige automáticamente la trazabilidad de paquetes y despliegues.

6. Construcción, Paquetes y Despliegue

El proceso actual de construcción, empaquetamiento y despliegue opera con alta intervención manual. Después del cierre de ciclo y la certificación, Desarrollo prepara los componentes requeridos, consolida entregables y entrega instrucciones técnicas para que Infraestructura o el cliente ejecuten la instalación según el modelo de despliegue aplicable.

La evidencia revisada en los procedimientos de Efinómina muestra que el despliegue no corresponde a una única aplicación simple, sino a un conjunto de componentes web, APIs, gateway, workflow, procesos y reportes que requieren configuración específica por ambiente. Este modelo es comparable al comportamiento operativo de Humano, donde la entrega depende de rutas, servidores, configuraciones, permisos, archivos y validaciones manuales.

Aspecto observado	Descripción	Impacto operativo
Construcción manual de componentes	La construcción de componentes no se evidencia como un proceso completamente automatizado y homogéneo para todos los productos, repositorios y tecnologías. En varios casos requiere recompilar manualmente proyectos o preparar artefactos específicos según el componente involucrado.	La generación del entregable depende de conocimiento técnico del desarrollador o líder, lo que incrementa el riesgo de inconsistencias entre lo desarrollado, lo probado y lo finalmente empaquetado.

Aspecto observado	Descripción	Impacto operativo
Despliegue por componente	El despliegue se realiza por componentes separados, como aplicación web, portal, APIs, gateway, workflow, procesos y reportes. Cada componente puede requerir rutas físicas, App Pools, puertos, archivos de configuración, permisos y validaciones particulares.	La entrega depende de conocimiento específico por componente y aumenta la probabilidad de diferencias entre ambientes, errores de configuración o instalaciones incompletas.
Preparación de paquetes ZIP	Los entregables se consolidan manualmente en paquetes ZIP para su posterior entrega a Infraestructura o al cliente. Estos paquetes pueden incluir componentes, archivos, instrucciones o ajustes requeridos según el ambiente, cliente o versión.	El empaquetamiento manual aumenta el riesgo de omitir archivos, incluir componentes incorrectos o generar diferencias entre paquetes destinados a distintos clientes o ambientes.
Configuración manual de IIS y servicios	Las guías de despliegue evidencian creación manual de sitios, Application Pools, rutas físicas, puertos, pipeline, permisos de carpeta y conversión de carpetas en aplicaciones dentro de IIS.	El éxito del despliegue depende de que cada paso sea ejecutado correctamente por el responsable operativo, lo que incrementa la exposición a errores humanos y reprocesos.
Configuración manual de archivos y rutas	Los componentes requieren ajustes en WebConfig, appsettings.json, Ocelot.json, archivos .config, rutas de logs, rutas de reportes, rutas de salida, cadenas de conexión y parámetros por ambiente.	La existencia de múltiples parámetros sensibles por componente dificulta estandarizar la entrega y aumenta el riesgo de fallas por rutas, permisos, puertos o cadenas de conexión incorrectas.
Prerrequisitos técnicos por ambiente	Los despliegues requieren validar condiciones previas como sistema operativo, IIS, versiones de .NET, Java, motor de base de datos, permisos de carpetas, puertos disponibles y configuración de servicios.	La falta de validación automática de prerrequisitos puede generar fallas durante la instalación o comportamientos distintos entre ambientes.
Instrucciones técnicas de entrega	Desarrollo prepara instrucciones técnicas para orientar la instalación del paquete. Estas instrucciones pueden incluir rutas, componentes, orden de ejecución, configuraciones o consideraciones particulares del despliegue.	La correcta instalación depende de la claridad de las instrucciones y de la interpretación de quien ejecuta el despliegue, lo que puede generar reprocesos si la información no es completa o suficientemente precisa.
Entrega mediante RFC o correo	La solicitud de instalación se formaliza mediante RFC, correo u otro canal operativo. A partir de esta comunicación, Infraestructura o el cliente reciben el paquete y las instrucciones para ejecutar el despliegue.	El flujo queda sujeto a coordinación manual entre áreas o con terceros, lo que puede introducir tiempos de espera, pérdida de contexto o brechas entre quien prepara el paquete y quien lo instala.

Aspecto observado	Descripción	Impacto operativo
Participación de Infraestructura	Infraestructura ejecuta despliegues cuando el ambiente está bajo responsabilidad interna o cuando se requiere apoyo operativo para instalar paquetes, habilitar ambientes o aplicar instrucciones técnicas.	La separación entre quien construye el paquete y quien ejecuta la instalación puede generar dependencia de comunicación detallada, validación cruzada y disponibilidad operativa de Infraestructura.
Despliegues ejecutados por clientes	En algunos modelos, el cliente recibe el paquete y realiza la instalación en sus propios ambientes de certificación o producción. Esto ocurre especialmente cuando la infraestructura pertenece al cliente o cuando el contrato define ese modelo de operación.	La fábrica pierde visibilidad directa sobre la ejecución final del despliegue, quedando dependiente de la retroalimentación del cliente para confirmar instalación, errores o resultados de validación.
Modelos de despliegue coexistentes	Conviven despliegues SaaS/PaaS, On-Premise y BPO o tercerización. Cada modelo puede tener responsables, pasos, ambientes, restricciones y niveles de control distintos.	La coexistencia de modelos obliga a preparar entregas con particularidades por cliente, contrato, infraestructura y ambiente, aumentando la complejidad operativa del delivery.
Ambientes de certificación y producción	Los paquetes pueden pasar por ambientes de certificación antes de avanzar a producción. En algunos casos, el cliente realiza sus propias pruebas y posteriormente decide el paso a producción.	La trazabilidad del estado final depende del modelo de operación. No siempre existe visibilidad completa y centralizada sobre qué versión quedó instalada, validada o promovida por cada cliente.
Validación post-despliegue manual	La validación se realiza mediante pruebas operativas como cargar la aplicación, probar login, verificar Swagger, validar token, revisar conexión a base de datos, revisar logs y confirmar estado del sitio, servicio o App Pool.	No se evidencia una validación automatizada centralizada del despliegue, por lo que la confirmación depende de chequeos manuales y criterio del responsable.
Errores recurrentes de instalación	Las guías documentan errores comunes asociados a permisos, configuración de IIS, lectura de WebConfig, rutas restrictivas, conexión a base de datos, Java, archivos faltantes o versiones incompatibles.	El proceso de despliegue está expuesto a fallas operativas conocidas, que pueden generar reprocesos, demoras y dependencia de personas con experiencia en la instalación.
Versionamiento visible del producto	No se evidencia un estampado automático y visible de la versión desplegada dentro del producto o del código instalado.	Se dificulta auditar con precisión qué versión, rama, paquete o conjunto de cambios quedó instalado en cada ambiente o cliente.
Trazabilidad entre paquete y cambio	No se evidencia una relación completamente automatizada entre Work Items, ramas, Pull Requests, versión, paquete ZIP, RFC y ambiente desplegado.	Se dificulta reconstruir de punta a punta qué requerimientos o bugs fueron incluidos en un paquete específico y dónde quedaron instalados.

Aspecto observado	Descripción	Impacto operativo
Fallos asociados a preparación manual	Los errores pueden estar relacionados con paquetes incompletos, componentes faltantes, instrucciones insuficientes, diferencias de ambiente o fallas humanas durante la preparación e instalación.	Se incrementa el riesgo de reprocesos, pases fallidos, demoras en la estabilización y pérdida de confianza en la entrega.

6.1 Hallazgo de Construcción, Paquetes y Despliegue

El proceso actual de construcción, empaquetamiento y despliegue es funcional, pero opera con alta dependencia de intervención manual, configuración específica por componente y coordinación entre Desarrollo, Infraestructura y clientes. La entrega no se evidencia como un flujo automatizado y trazable de punta a punta. En la práctica, depende de paquetes ZIP, instrucciones técnicas, RFC, configuración manual de IIS, archivos de parámetros, rutas, permisos, puertos y validaciones posteriores a la instalación.

Esta condición incrementa el riesgo de errores por componentes faltantes, diferencias entre ambientes, instrucciones incompletas o pérdida de trazabilidad entre lo certificado y lo finalmente instalado, especialmente cuando el cliente ejecuta el despliegue y la fábrica pierde visibilidad directa sobre el resultado final. En consecuencia, el delivery actual permite sostener la operación, pero con baja automatización, trazabilidad parcial de versiones y alta dependencia de personas clave para asegurar que cada paquete quede correctamente instalado en el ambiente esperado.

7. Estado actual de Azure DevOps

Azure DevOps se utiliza como herramienta operativa para registrar, consultar y hacer seguimiento al trabajo del proyecto Sistema de Información Humano. La configuración observada corresponde a un proceso Agile customizado, no a una implementación Scrum pura ni a un modelo CMMI. Esta configuración conserva elementos base como Epic, Feature, User Story, Bug y Task, e incorpora tipos de trabajo adicionales para análisis, avance, calidad, configuración y pruebas.

La evidencia revisada muestra que Azure DevOps permite operar el flujo diario del equipo, pero presenta limitaciones para gobernar de forma clara la planeación, el seguimiento visual, la separación entre desarrollo y soporte, la trazabilidad de versiones y la medición diferenciada del trabajo.

Dimensión observada	Evidencia disponible	Lectura operativa AS-IS
Proceso configurado	El proyecto utiliza un proceso Agile customizado. No corresponde a Scrum puro, CMMI ni Agile estándar sin modificaciones.	Azure DevOps está adaptado a la operación actual del equipo, pero la customización introduce dependencias manuales para representar el flujo real de trabajo.
Tipos base de trabajo	Se identifican Epic, Feature, User Story, Bug y Task como tipos base del proceso.	La herramienta conserva una estructura compatible con planeación, backlog, desarrollo, defectos y tareas operativas.
Tipos de trabajo adicionales	Se identifican tipos adicionales para análisis de desarrollo, avance, calidad, configuración y pruebas.	El equipo intenta representar actividades específicas mediante tipos de trabajo adicionales, lo que aumenta la necesidad de disciplina operativa para mantener la información consistente.

Dimensión observada	Evidencia disponible	Lectura operativa AS-IS
Niveles de backlog	El backlog principal se organiza en Epics, Features, Stories y Tasks. Otros tipos, como Issue y tareas customizadas, existen en el proceso, pero no necesariamente están asociados directamente al backlog o tablero principal.	El backlog visible no representa necesariamente todo el trabajo existente en el proyecto. Esto afecta la lectura completa de la demanda y del trabajo operativo.
Issues	El tipo Issue aparece como tipo de trabajo existente, pero está configurado sin backlog asociado. La evidencia indica que queda visible en el inventario general, pero no entra al backlog ni al tablero principal.	Parte del trabajo registrado puede quedar por fuera de la planeación visible y del flujo operativo principal, afectando la trazabilidad y priorización diaria.
Tablero operativo	El tablero opera con los estados New, Active, Unresolved, Resolved, Rejected, Ended y Closed.	El tablero permite mover ítems y visualizar avance general, pero sus estados son genéricos y no describen con precisión etapas operativas como análisis, desarrollo, calidad, validación funcional y cierre.
Trabajo en curso	Se observa acumulación de 34 ítems en Active frente a un límite configurado de 5.	Existe exceso de trabajo en curso visible en el tablero. Esto sugiere que hay más trabajo iniciado que capacidad efectiva de cierre, lo que puede afectar foco, predictibilidad y tiempo de ciclo.
Histórico visible	Se observa una acumulación superior a 300 ítems en Rejected.	El tablero mezcla trabajo activo con histórico inactivo o rechazado, lo que puede contaminar la lectura diaria del flujo y dificultar la gestión visual.
Desarrollo y soporte	La evidencia indica que bugs productivos y nuevos desarrollos comparten el mismo nivel visual y conviven en la misma lista de trabajo.	La operación mezcla evolución de producto, soporte, correcciones y errores productivos en un mismo frente visual. Esto limita la medición diferenciada entre mantenimiento y desarrollo nuevo.
Campos de clasificación	La información disponible permite consultar tipo de trabajo, estado, responsable, creador, área, iteración, etiquetas, padre, proyecto, solicitante y módulo.	Existe información suficiente para construir reportes por tipo de trabajo, responsable, estado, área, iteración, proyecto, solicitante y módulo. El reto está en usar estos campos de forma consistente para separar mejor la operación.

Dimensión observada	Evidencia disponible	Lectura operativa AS-IS
Clasificación de bugs	La evidencia indica que un bug no debe clasificarse automáticamente como soporte, porque puede corresponder a desarrollo, soporte o calidad según su origen, área, iteración, etiquetas, responsable, solicitante, módulo o relación con otros elementos.	La medición de bugs requiere criterios de clasificación. De lo contrario, se puede distorsionar la lectura entre defectos de desarrollo, incidencias de producción, hallazgos de calidad y ajustes operativos.
Relación padre-hijo	La información disponible permite analizar relaciones entre épicas, funcionalidades, historias, bugs, tareas, issues y tipos de trabajo adicionales.	Hay capacidad para analizar jerarquía y trazabilidad funcional entre elementos, siempre que las relaciones estén correctamente registradas en Azure DevOps.
Iteración y versión	La evidencia indica que las iteraciones bimensuales se usan como versión esperada y que el nombre de la iteración se relaciona con la rama Git de despliegue. También se indica que el número de versión no queda asociado a metadata de build, tag o artefacto versionado.	La versión se gestiona principalmente como convención operativa asociada a la iteración y a la rama, no como trazabilidad técnica completa hacia build, artefacto y despliegue.
Daily y gestión verbal	Se observa una daily aproximada de 60 minutos con 24 personas. La evidencia relaciona esta duración con la falta de estados visuales específicos en el tablero.	La daily compensa información que el tablero no comunica de forma clara. Esto convierte parte de la sincronización en reporte verbal de estado.
Métricas diferenciadas	La estructura actual unifica soporte técnico y desarrollo en un solo frente, lo que limita medir el tiempo invertido en mantenimiento frente a nuevos desarrollos.	Azure DevOps contiene datos para análisis, pero la configuración actual dificulta separar capacidad dedicada a operación, soporte, calidad, deuda y evolución del producto.

7.1 Hallazgo del estado actual de Azure DevOps

Azure DevOps es una herramienta central en la operación del proyecto Sistema de Información Humano. La evidencia muestra un proceso Agile customizado, tipos de trabajo base y adicionales, columnas de tablero definidas, campos de clasificación y datos suficientes para consultar información de planeación, desarrollo, soporte, calidad y seguimiento.

Sin embargo, la configuración actual permite operar mejor de lo que permite gobernar. El backlog visible no representa necesariamente todo el trabajo real, los Issues quedan fuera del tablero principal, los bugs productivos y los nuevos desarrollos comparten el mismo nivel visual, y los estados del tablero no reflejan con claridad etapas operativas como análisis, desarrollo, calidad, validación funcional y cierre.

Esta configuración reduce la claridad gerencial del flujo. El tablero permite mover elementos entre estados, pero no comunica de forma suficientemente simple qué está en curso, qué está bloqueado, qué está pendiente de validación y qué puede considerarse cerrado. Por eso, parte del avance debe reconstruirse mediante explicación verbal en la daily, reportes complementarios o conocimiento específico del equipo.

El hallazgo principal es que el problema no es la ausencia de Azure DevOps, sino la forma en que está configurado y usado. La mezcla de tipos de trabajo, el exceso de trabajo en Active, el histórico visible en Rejected y la trazabilidad de versión soportada principalmente en la convención de iteración y rama limitan su capacidad para gobernar la operación, medir capacidad real y sostener decisiones gerenciales con información limpia y diferenciada.

8. Estado actual de repositorios y ramas

El levantamiento técnico evidencia que el código fuente del ecosistema Humano se encuentra distribuido en múltiples proyectos, repositorios y ramas dentro de Azure DevOps. Esta estructura refleja la evolución histórica del producto, la coexistencia de varios frentes funcionales y la necesidad de mantener componentes diferenciados para Humano, Efinómina, Carrera, Humano API, reportes y otros módulos relacionados.

La mayor concentración se observa en el proyecto Sistema de Información Humano, que agrupa 14 repositorios y 2.791 ramas. Dentro de este proyecto, el repositorio Humano concentra 2.525 ramas, lo que lo convierte en el principal foco de acumulación histórica de ramas de desarrollo, bugs, features, fixes, ciclos, despliegues y versiones. El inventario también identifica Humano API como un proyecto independiente, con un repositorio propio y 120 ramas.

Dimensión observada	Evidencia disponible	Lectura operativa AS-IS
Distribución de proyectos	Se identifican 41 proyectos en Azure DevOps, cada uno con Board asociado. Algunos proyectos tienen repositorios y otros no.	La organización cuenta con varios contenedores de trabajo, pero la operación técnica relevante se concentra en pocos proyectos con repositorios activos.
Proyecto principal	El proyecto Sistema de Información Humano concentra 14 repositorios y 2.791 ramas.	Este proyecto actúa como contenedor técnico principal del ecosistema Humano y agrupa múltiples componentes relacionados.
Repositorio Humano	Dentro de Sistema de Información Humano, el repositorio Humano registra 2.525 ramas.	El repositorio core concentra la mayor carga histórica de cambios, versiones, bugs, features, fixes y ramas de despliegue.
Repositorios complementarios	Se identifican repositorios como Efinomina, EfinominaAPI, Efinomina-reportes, HumanoAPP, HumanoEPJava, Humano-Reportes, Carrera_Calificacion y Carrera_Seleccion.	El producto no se gestiona como una sola base de código; requiere coordinación entre varios repositorios y componentes técnicos.
Humano API	Humano API aparece como proyecto independiente con el repositorio Humano_API y 120 ramas.	El backend/API tiene una administración separada del proyecto principal, lo que introduce un frente adicional de control de versiones y coordinación técnica.
Convenciones de ramas	Se observan ramas con prefijos y nombres como feature, bug, fix, pbi, user, version, despliegue, despliegueCiclo, humano-develop, humano-fix, certification y nombres de personas o proveedores.	No se observa una convención única y homogénea para nombrar ramas. La nomenclatura refleja mezcla de propósito técnico, versión, cliente, persona, proveedor, ciclo o corrección.

Dimensión observada	Evidencia disponible	Lectura operativa AS-IS
Ramas de versión y despliegue	Existen ramas asociadas a versiones y despliegues, por ejemplo ramas de tipo version/* y de-despliegue/produccion_humano_version/*	La versión se materializa principalmente como rama o convención de nombre, no necesariamente como artefacto técnico versionado de forma integral.
Ramas históricas	El volumen de ramas acumuladas, especialmente en Humano, evidencia permanencia de ramas antiguas o paralelas.	La falta de depuración sostenida incrementa la complejidad para buscar ramas vigentes, entender el origen de cambios y controlar integraciones.
Ramas por persona o proveedor	Se observan ramas asociadas a usuarios, proveedores o nombres propios.	Parte del trabajo queda reflejado en ramas personalizadas, lo que puede dificultar gobierno, estandarización y transferencia de conocimiento.
Ramas por bug, feature o PBI	Existen ramas que referencian bugs, features, PBIs, HUs o identificadores funcionales.	Hay intención de relacionar ramas con trabajo específico, pero la variedad de convenciones dificulta una lectura uniforme y automatizable.
Trazabilidad de versión	La evidencia indica que la iteración se usa como versión esperada y que el nombre de la iteración puede corresponder a la rama Git. También se indica que el número de versión no queda asociado a archivo de configuración, metadata de build, tag o artefacto versionado.	La trazabilidad entre versión, código fuente, build, artefacto y despliegue no queda completamente cerrada desde el control de versiones.

8.1 Radiografía de repositorios principales

Proyecto / repositorio	Lectura AS-IS	Ramas identificadas
Sistema de Información Humano	Proyecto principal del ecosistema Humano. Agrupa repositorios asociados a core, reportes, Efinómina, Carrera, APP, Java y pruebas de pipelines.	2.791
Humano	Repositorio core. Concentra la mayor acumulación de ramas históricas, técnicas, funcionales, de bugs, features, fixes, ciclos, versiones y despliegues.	2.525
Efinomina	Repositorio asociado a Efinómina dentro del proyecto Sistema de Información Humano.	80

Proyecto / repositorio	Lectura AS-IS	Ramas identificadas
Efinomina-reportes	Repositorio de reportes de Efinómina.	132
Humano-Reportes	Repositorio de reportería asociado al ecosistema Humano.	37
Carrera_Calificacion y Carrera_Seleccion	Conjunto de repositorios relacionados con componentes de Carrera, incluyendo web, reportes, API y transferencia.	8
HumanoAPP y HumanoEPJava	Componentes complementarios del ecosistema Humano.	6
Humano API / Humano_API	Proyecto independiente con repositorio propio para API.	120

8.2 Hallazgo del estado actual de repositorios y ramas

El ecosistema Humano cuenta con repositorios diferenciados y una estructura de ramas que permite sostener múltiples frentes de desarrollo, mantenimiento, reportería, API, Efinómina, Carrera y componentes complementarios. Sin embargo, la evidencia muestra una alta concentración de ramas en el proyecto Sistema de Información Humano y, especialmente, en el repositorio Humano.

El principal hallazgo no es la falta de control de versiones, sino la acumulación y dispersión de ramas. La coexistencia de ramas por feature, bug, fix, PBI, usuario, proveedor, versión, certificación y despliegue dificulta identificar con rapidez qué ramas están vigentes, cuáles son históricas, cuáles representan una versión liberada y cuáles contienen cambios pendientes o parciales.

Adicionalmente, la versión se apoya principalmente en convenciones de nombre de iteración y rama. Al no quedar asociada de forma clara a metadata de build, tag o artefacto versionado, se limita la trazabilidad técnica entre código fuente, versión generada, paquete entregado y despliegue final.

En consecuencia, los repositorios permiten operar el desarrollo, pero el volumen de ramas, la diversidad de nomenclaturas y la trazabilidad parcial de versiones reducen la claridad para gobernar integraciones, depurar histórico, controlar releases y auditar qué código corresponde exactamente a cada entrega.

9. Hallazgos consolidados

El análisis AS-IS evidencia que la operación actual de desarrollo permite sostener el negocio, atender requerimientos, corregir errores y liberar versiones. Sin embargo, esta operación se apoya en una combinación de conocimiento histórico, coordinación manual, herramientas parcialmente gobernadas y prácticas técnicas que han crecido por acumulación.

Los hallazgos consolidados no muestran ausencia de operación ni falta total de herramientas. Por el contrario, muestran una organización que funciona, pero con limitaciones importantes para escalar, medir capacidad real, reducir reprocesos, controlar versiones y sostener trazabilidad extremo a extremo.

Frente analizado	Hallazgo consolidado	Impacto AS-IS
Operación y capacidad	Desarrollo, Calidad e Infraestructura actúan como equipos transversales que atienden requerimientos, bugs, soporte, urgencias y despliegues sobre múltiples productos y clientes.	La capacidad disponible se consume en varios frentes simultáneos, lo que dificulta separar esfuerzo de evolución, mantenimiento, soporte y urgencias.

Frente analizado	Hallazgo consolidado	Impacto AS-IS
Gestión de demanda	La demanda se registra principalmente en Azure DevOps, pero parte de la clasificación, priorización, seguimiento y control operativo depende de criterios manuales, convenciones de título o archivos externos.	La trazabilidad de la demanda no queda completamente centralizada, lo que limita la lectura gerencial por cliente, producto, frente, módulo o tipo de trabajo.
Ciclo de desarrollo	El ciclo actual opera por ventanas amplias de trabajo, pruebas beta, validación de ciclo y certificación, con atención paralela de bugs críticos o urgencias productivas.	El flujo permite organizar entregas, pero reduce flexibilidad para liberar cambios a voluntad y puede extender los tiempos de respuesta ante solicitudes o defectos.
Código y mantenibilidad	Humano opera como un ecosistema técnico amplio, compuesto por múltiples soluciones, módulos, capas, componentes transversales, servicios, reportería y tecnologías de distintas generaciones.	La modificación segura del producto exige conocimiento histórico, análisis de dependencias y validación cuidadosa de impacto entre módulos.
Acoplamiento técnico	FOMAG, Efinómina, Carrera, reportes, APIs y componentes satélites conviven parcial o totalmente dentro del ecosistema técnico y operativo de Humano.	La separación entre producto core, soluciones específicas y componentes complementarios no siempre es limpia, lo que aumenta el riesgo de impacto colateral.
Patrones de implementación	Se observan patrones de código con potencial impacto en rendimiento y mantenibilidad, como consultas repetitivas dentro de ciclos, lógica distribuida entre aplicación y base de datos, y dependencia de componentes compartidos.	En procesos masivos, estos patrones pueden amplificar tiempos de respuesta, presión sobre base de datos y dificultad de diagnóstico.
Gobierno del código	No se evidencia un estándar institucional homogéneo y verificable de escritura, organización, revisión y calidad técnica del código.	La mantenibilidad depende de criterio individual, revisión manual y conocimiento histórico, dificultando control automático de calidad y evolución ordenada.
Propagación de deuda técnica	Varias soluciones y componentes evidencian crecimiento por copia, adaptación y variación del producto base y sus estructuras.	La deuda técnica no queda contenida en un solo producto; se distribuye en módulos, reportes, APIs, reglas, consultas y configuraciones derivadas.
Construcción y paquetes	La construcción, preparación de paquetes y entrega técnica dependen en buena medida de actividades manuales, archivos ZIP, instrucciones operativas y coordinación entre Desarrollo e Infraestructura.	Aumenta el riesgo de paquetes incompletos, errores de instalación, reprocesos y diferencias entre lo desarrollado, lo probado y lo desplegado.

Frente analizado	Hallazgo consolidado	Impacto AS-IS
Despliegue	El despliegue se coordina mediante pasos operativos y comunicación formal, con intervención de Infraestructura y validaciones posteriores.	La separación entre quien prepara el cambio y quien lo instala exige alta precisión en instrucciones, artefactos y comunicación.
Azure DevOps	Azure DevOps sí existe, se usa y contiene información relevante, pero su configuración actual permite operar mejor de lo que permite gobernar.	Los estados genéricos, la mezcla de trabajo y la visibilidad parcial de algunos tipos de elementos limitan la lectura gerencial del flujo.
Repositorios y ramas	El ecosistema cuenta con múltiples repositorios y una alta acumulación de ramas, especialmente en el proyecto Sistema de Información Humano y en el repositorio Humano.	La diversidad de ramas por feature, bug, fix, usuario, proveedor, ciclo, versión y despliegue dificulta gobernar integraciones, depurar histórico y auditar releases.
Versionamiento	La versión se soporta principalmente en convenciones de nombre de iteración y rama, sin evidenciar una asociación completa con metadata de build, tag o artefacto versionado.	Se limita la trazabilidad técnica entre código fuente, versión generada, paquete entregado y despliegue final.
Métricas y gobierno	Existen datos suficientes para análisis, pero no siempre están normalizados, centralizados o estructurados para medición automática.	Las decisiones gerenciales requieren consolidación manual o interpretación del equipo, lo que reduce oportunidad, precisión y comparabilidad de las métricas.

9.1 Lectura consolidada AS-IS

La compañía cuenta con una operación funcional y con herramientas relevantes para gestionar desarrollo, seguimiento y control de código. No obstante, el modelo actual presenta una alta dependencia de coordinación manual, conocimiento experto y convenciones internas.

El principal patrón observado es la acumulación: acumulación de demanda, ramas, variantes de código, reportes, módulos, reglas operativas y mecanismos paralelos de seguimiento. Esta acumulación ha permitido sostener la operación en el tiempo, pero también ha incrementado la complejidad para gobernar el flujo de punta a punta.

En síntesis, el AS-IS muestra una fábrica de software operativa, pero con retos claros de gobierno técnico y operativo: trazabilidad fragmentada, separación insuficiente entre soporte y evolución, versionamiento débilmente formalizado, alta carga manual en despliegues y dificultad para medir capacidad real con información limpia y diferenciada.

10. Conclusión AS-IS

La situación actual evidencia una organización de desarrollo que ha logrado sostener productos críticos para el negocio mediante experiencia del equipo, conocimiento histórico, adaptación operativa y uso de herramientas como Azure DevOps y repositorios Git. La operación existe, funciona y permite responder a requerimientos, bugs, soporte, ciclos de entrega y necesidades de clientes.

Sin embargo, el modelo actual presenta señales claras de desgaste operativo y complejidad acumulada. La gestión de demanda, el ciclo de desarrollo, la estructura del código, la preparación de paquetes, el

despliegue, el uso de Azure DevOps y el gobierno de ramas no conforman todavía un flujo completamente integrado, medible y trazable de extremo a extremo.

El principal reto no es reemplazar la operación existente, sino ordenarla y fortalecerla. La compañía no parte de cero: cuenta con equipos, conocimiento, productos, repositorios, tableros, reportes y prácticas instaladas. El desafío está en reducir dependencia manual, simplificar el flujo, separar mejor los frentes de trabajo, formalizar versionamiento, mejorar la trazabilidad y convertir la información existente en capacidad real de gobierno.

Un punto adicional relevante es que la complejidad del código no debe confundirse con la carga pendiente, la acumulación de ramas o la construcción manual de productos. El backlog acumulado refleja presión de demanda y capacidad; las ramas acumuladas reflejan debilidad en gobierno de configuración; el código heterogéneo refleja deuda técnica y ausencia de estándares verificables; y la construcción manual refleja baja automatización del delivery. Aunque estos elementos se relacionan, su naturaleza y tratamiento son distintos.

Desde la perspectiva AS-IS, Humano y sus soluciones relacionadas operan como un ecosistema técnico amplio, maduro y funcional, pero con alta dependencia de personas clave, convenciones internas y coordinación manual. Esto limita la escalabilidad de la fábrica de software, incrementa el riesgo de reprocesos y dificulta sostener decisiones gerenciales con métricas limpias, oportunas y comparables.

La conclusión central es que la organización tiene una base sólida para evolucionar, pero requiere pasar de una operación sostenida por conocimiento y esfuerzo manual a un modelo más gobernado, trazable y predecible. El foco de mejora debe estar en fortalecer el control del flujo completo: desde la demanda y el desarrollo, hasta la integración, pruebas, paquetes, despliegue, versionamiento y medición de capacidad.

Ajuste complementario — Flujo real de demanda, refinamiento y seguimiento operativo

A. Flujo actual real de ingreso y refinamiento de casos

La operación actual no parte de una ausencia total de refinamiento funcional. El flujo observado debe entenderse así:

1. Cada jefe de línea de negocio solicita a sus colaboradores cargar casos en Azure DevOps.
2. La demanda puede provenir de varias líneas que impactan un mismo producto, especialmente HU.
3. Antes del inicio del ciclo, cada jefe de línea se reúne con Jenny.
4. Jenny, en rol BA / QA, filtra, depura y refina las historias.
5. Jenny participa en la estructuración funcional de las HU, criterios y calidad de la definición.
6. El líder de desarrollo participa en la estimación, factibilidad y lectura técnica.
7. Con base en ese refinamiento se define qué podría entrar al ciclo.

Este flujo evidencia que QA / BA sí participa antes del desarrollo. La brecha no está en la inexistencia de refinamiento, sino en su alto nivel de dependencia manual, en la concentración del proceso sobre Jenny y en la entrada distribuida de demanda desde varias líneas.

B. Riesgo de duplicidad y solapamiento de historias

Cuando varios jefes de línea de negocio cargan solicitudes sobre un mismo producto o componente, pueden generarse historias repetidas, solapadas o incluso contradictorias. Este riesgo es especialmente visible en productos compartidos como Humano, donde varias líneas pueden solicitar ajustes sobre capacidades similares.

Antes de comprometer una HU al ciclo, debería validarse:

- Si ya existe una historia similar.
- Si la solicitud se solapa con otra línea de negocio.

- Si el requerimiento puede consolidarse con otra HU.
- Qué línea de negocio tiene prioridad.
- Quién será responsable funcional de aceptación.
- Qué producto, módulo, cliente o frente queda afectado.

C. Precisión sobre Azure DevOps, Excel, chat y reuniones

Azure DevOps sí funciona como plataforma formal de registro para buena parte de la demanda, especialmente historias, bugs y tareas. Sin embargo, el seguimiento operativo no está completamente centralizado ni normalizado.

La operación se complementa con Excel, chats, correos y reuniones para resolver aspectos que no siempre quedan estructurados en la herramienta, como priorización fina, seguimiento por cliente, coordinación de urgencias, validaciones puntuales, acuerdos operativos o consolidaciones manuales.

La formulación correcta del hallazgo es:

Azure DevOps se usa, pero no absorbe todavía todo el gobierno operativo.

La brecha está en la fragmentación del seguimiento y en la dependencia de controles paralelos.

D. Impacto del ciclo de dos meses

El ciclo aproximado de dos meses permite agrupar una cantidad amplia de trabajo, pero reduce la capacidad de adaptación controlada frente a cambios de prioridad y urgencias productivas.

Durante el ciclo pueden ocurrir los siguientes efectos:

- Prioridades que cambian después de haber planeado el ciclo.
- Incendios operativos que desplazan trabajo planificado.
- Historias que se inician pero no se terminan.
- Funcionalidades que quedan desarrolladas parcialmente.
- Validaciones QA que pueden no completarse con la profundidad esperada.
- Dificultad para diferenciar trabajo comprometido, trabajo interrumpido y trabajo realmente terminado.

E. Hallazgo ajustado

La organización cuenta con Azure DevOps y con una práctica de refinamiento funcional previa lanzamiento.

No obstante, la demanda entra distribuida desde varias líneas de negocio, puede duplicarse sobre la misma funcionalidad.

La brecha principal está en el gobierno de demanda, la estabilidad del compromiso, la consolidación de la demanda.